PAYROLL MANAGEMENT SYSTEM

COURSE NAME : OBJECT-ORIENTED PROGRAMMING

PROJECT TITLE : PAYROLL MANAGEMENT SYSTEM

STUDENT NAME : SURANJANA MAJUMDAR

ENROLLMENT NUMBER : 12024052004001

ROLL NUMBER : 02

INSTRUCTOR NAME : DR. SUSOVAN JANA, DR. SOUMADIP BISWAS

SEMESTER : SPRING 2026

1. PROBLEM STATEMENT :

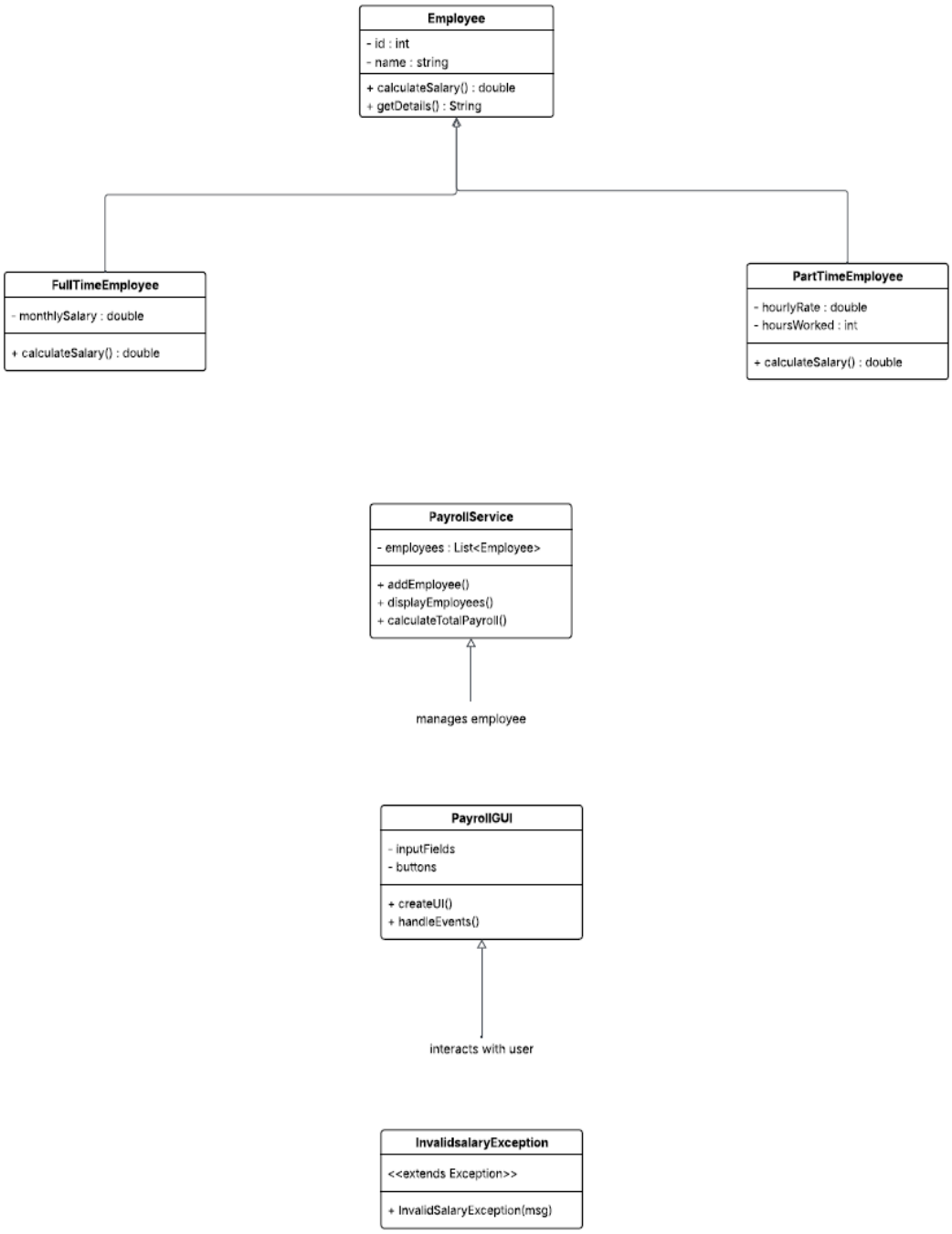
Managing employee payroll manually is a time-consuming and error-prone task. Organizations often employ different categories of workers such as full-time employees who receive fixed monthly salaries and part-time employees who are paid based on hours worked. Calculating salaries manually for different employee types can lead to incorrect computations, poor record management, and lack of transparency.

The objective of this project is to design and implement a **Payroll Management System using Java and Object-Oriented Programming concepts**. The system automates employee data management and salary calculations based on employment type. It allows the user to add employee records, compute payroll automatically, and display payroll summaries. The system also includes error handling to manage invalid inputs and ensure program stability.

By using modular design and proper OOP principles, the system becomes scalable, maintainable, and easy to extend for future organizational needs.

2. SYSTEM DESIGN :

- **UML DESIGN**



- **DESCRIPTION OF MAJOR CLASS**

  Employee (Abstract Class)
  Defines common attributes such as employee ID and name. It also declares the salary calculation method which will be implemented by subclasses.

  FullTimeEmployee
  Represents employees who receive a fixed monthly salary.

  PartTimeEmployee
  Represents employees paid based on hourly rate and hours worked.

  PayrollService
  Handles employee storage and payroll calculations.

  PayrollGUI
  Provides the graphical interface using Java Swing to interact with the system.

  InvalidSalaryException
  Handles invalid salary input values.

- **PACKAGE STRUCTURE**

```
src
└─ payroll
   ├─ Employee.java
   ├─ FullTimeEmployee.java
   ├─ PartTimeEmployee.java
   ├─ PayrollService.java
   ├─ PayrollGUI.java
   └─ InvalidSalaryException.java
```

- **INTERACTION FLOW**

  i. User opens the Payroll GUI.
  ii. User selects employee type (Full-time / Part-time).
  iii. User enters employee details.
  iv. System calculates salary based on employee type.
  v. Employee record is stored.
  vi. Payroll information is displayed.

3. OOP CONCRPTS DEMONSTRATED :

- **ABSTRACTION**

  Where Used:
  Employee abstract class.
  Why Used:
  To define a general blueprint for all employee types.
  Code Snippet:

```
public abstract class Employee {
protected int id;
protected String name;

public Employee(int id, String name) {
this.id = id;
this.name = name;
}

public abstract double calculateSalary();
}
```

  Explanation:
  The abstract class defines common properties of employees while forcing subclasses to implement their own salary calculation logic.

- **INHERITANCE**

  Where Used:
  FullTimeEmployee and PartTimeEmployee extend Employee.
  Why Used:
  To reuse common properties like ID and Name.
  Code Snippet:

```
public class FullTimeEmployee extends Employee {
private double monthlySalary;

public FullTimeEmployee(int id, String name, double salary) {
super(id, name);
this.monthlySalary = salary;
}
}
```

  Explanation:
  Inheritance allows specialized employee classes to reuse the attributes and methods of the base class.

- **POLYMORPHISM**

  Where Used:
  Salary calculation method.

Why Used:
Different employee types calculate salary differently.
Code Snippet:

```
public double calculateSalary() {
   return monthlySalary;
}
```

Explanation:
The same method calculateSalary() behaves differently depending on the object type**.**

- **EXCEPTION HANDLING**

  Where Used:
  Validating salary inputs.
  Why Used:
  To prevent system crashes when invalid input is given.
  Code Snippet:

```
try {
   double salary = Double.parseDouble(salaryField.getText());
} catch(NumberFormatException e) {
   JOptionPane.showMessageDialog(null, "Invalid Salary Input");
}
```

  Explanation:
  Try-catch blocks capture invalid inputs and display user-friendly error messages.

4. IMPLEMENTATION HIGHLIGHTS :

- **CREATING EMPLOYEE OBJECTS**

```
Employee emp = new FullTimeEmployee(id, name, salary);
payrollService.addEmployee(emp);
```

  This demonstrates polymorphism where the Employee reference can store objects of different employee types.

- **HOURLY SALARY CALCULATION**

```
public double calculateSalary() {
return hoursWorked * hourlyRate;
}
```

  Implements salary computation logic for part-time employees.

- **EMPLOYEE TYPE SELCTION USING UI**

```
if(fullTimeButton.isSelected()){
salaryField.setEnabled(true);
hoursField.setEnabled(false);
}
```

Improves usability by enabling only relevant input fields.

5. TESTING AND ERROR HANDLING :

- **TEST CASES**

Test Case	Input	Expected Output
Full-time employee	Salary entered	Salary displayed correctly
Part-time employee	Hours × rate	Correct salary calculated
Invalid input	Text instead of number	Error message

- **EDGE CASES**

i. Negative salary values.
ii. Empty fields.
iii. Invalid numeric inputs.

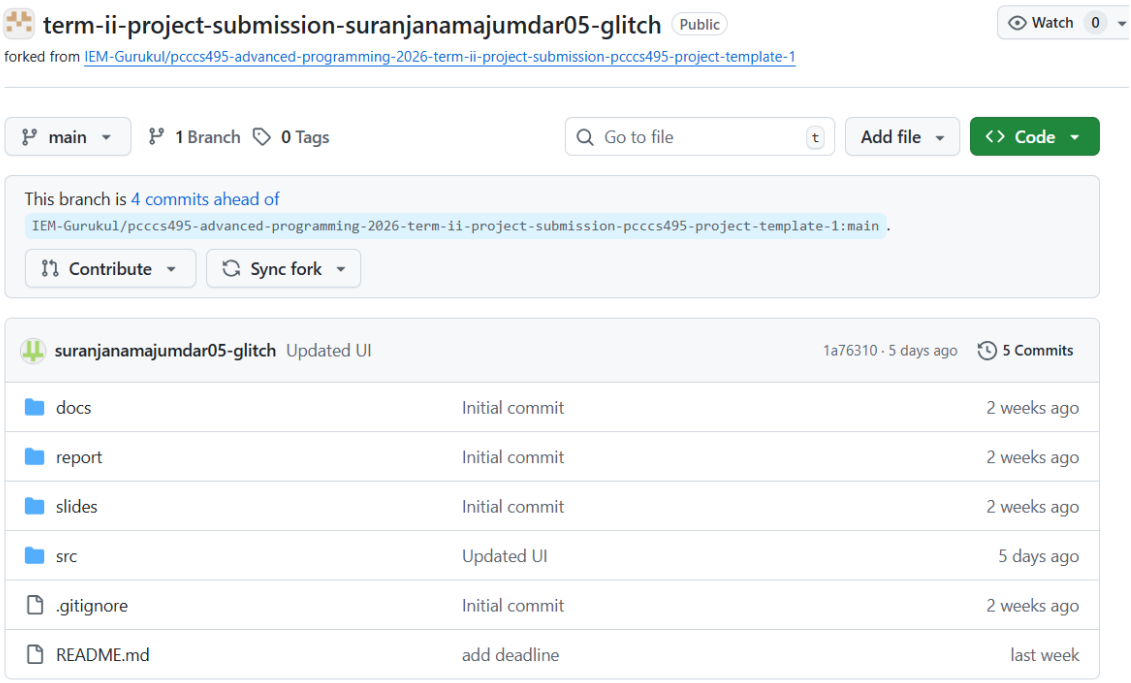
- **FAILURE SCENARIOS**

i. Invalid salary input.
ii. Missing employee details.
iii. Incorrect employee type selection.

6. GIT WORKFLOW

Typical workflow:

```
git add .
git commit -m "Updated UI"
git push
```



After these 5 commits :

i. Presentation slides updated.
ii. Required report is being uploaded.
iii. README files updated.

So, the number of commits increased.

7. CONCLUSION & FUTURE SCOPE :

The Payroll Management System successfully demonstrates the use of object-oriented programming principles such as abstraction, inheritance, polymorphism, and exception handling. The system automates employee payroll calculation and provides a graphical interface for easy interaction.

Future improvements may include:

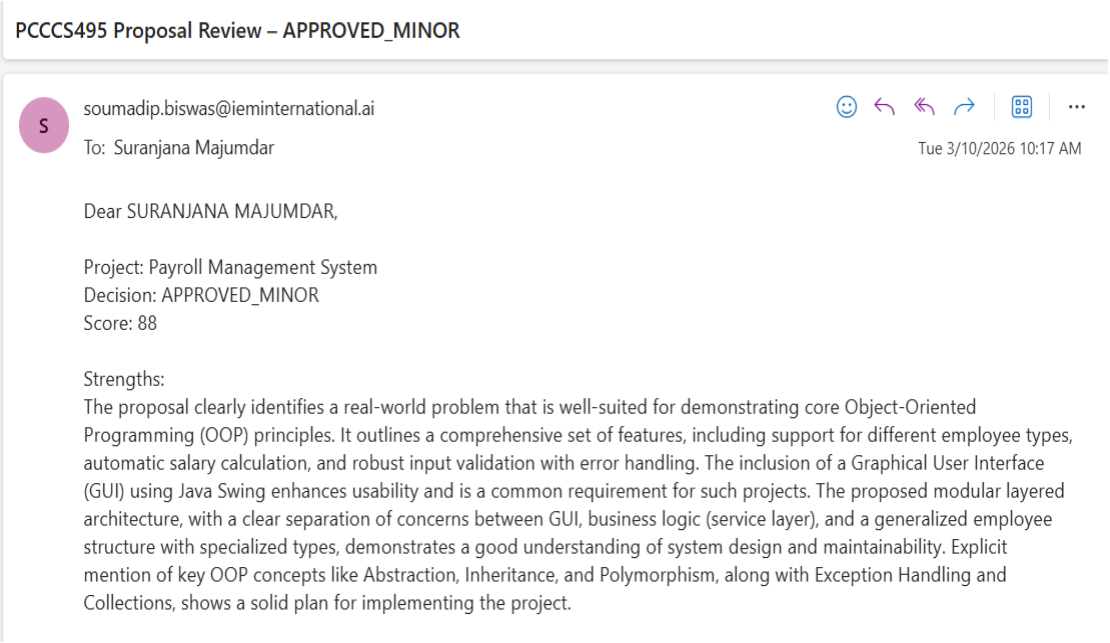
i. Employee login system.
ii. Salary slip generation.
iii. Web-based payroll system.

8. APPENDIX :

- **INITIAL PROJECT PROPOSAL**

  The Payroll Management System follows a modular layered architecture. A generalized employee structure defines common attributes and behaviors, while specialized employee types extend it and implement their own salary calculation methods. A service layer manages employee records, payroll calculations, and data storage using dynamic collections. The user interacts with the system through a **Graphical User Interface built using Java Swing**, which allows entry of employee details, selection of employee type, and viewing of payroll reports. The main execution layer connects the GUI with the business logic and handles errors using exception handling. This separation of layers improves system organization, maintainability, and scalability while following object-oriented programming principles.

- **REVIEW MAIL RECEIVED**



PCCCS495 Proposal Review – APPROVED_MINOR

S soumadip.biswas@ieminternational.ai

To: Suranjana Majumdar

Tue 3/10/2026 10:17 AM

Dear SURANJANA MAJUMDAR,

Project: Payroll Management System
Decision: APPROVED_MINOR
Score: 88

Strengths:
The proposal clearly identifies a real-world problem that is well-suited for demonstrating core Object-Oriented Programming (OOP) principles. It outlines a comprehensive set of features, including support for different employee types, automatic salary calculation, and robust input validation with error handling. The inclusion of a Graphical User Interface (GUI) using Java Swing enhances usability and is a common requirement for such projects. The proposed modular layered architecture, with a clear separation of concerns between GUI, business logic (service layer), and a generalized employee structure with specialized types, demonstrates a good understanding of system design and maintainability. Explicit mention of key OOP concepts like Abstraction, Inheritance, and Polymorphism, along with Exception Handling and Collections, shows a solid plan for implementing the project.